



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Gerber Mariano Pérez Gómez **Carné:** 1165825

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Es una interfaz que permite que dos sistemas se comuniquen a través de HTTP siguiendo los principios de la arquitectura REST. En dicha arquitectura, todo se modela como recursos que se identifican como URLs y se manipulan con HTTP estándar.

2. ¿Qué es un endpoint?

Es el punto de acceso concreto de una API donde se puede realizar una operación sobre un recurso. Combina una ruta con un método HTTP.

3. ¿Qué diferencia existe entre una URL y un endpoint?

La URL es la dirección completa y general de un recurso en la web. El endpoint es un concepto más específico: es la URL de la API junto con el método HTTP y la operación que ejecuta. La misma URL puede representar varios endpoints distintos.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

- HTTP GET: Consultar o leer datos, sin modificarlos.
- POST: crea un nuevo recurso.
- PUT: reemplaza un recurso completo.
- PATCH: modifica de forma parcial un recurso
- DELETE: elimina un recurso.

5. ¿Qué es JSON y por qué es común en las APIs?

JSON (JavaScript Object Notation) es un formato de texto ligero para representar datos con pares clave-valor, objetos y listas, es común porque es fácil de leer para las personas, liviano en comparación con XML, independiente del lenguaje y se integra de forma natural con JavaScript y con la web.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es un número de tres dígitos que el servidor incluye en la respuesta para indicar el resultado de la petición.

- 200 OK: la petición se procesó de forma correcta.
- 404 NOT FOUND: el recurso solicitado no existe.
- 500 INTERNAL SERVER ERROR: ocurrió un fallo inesperado en el servidor.



7. ¿Qué información se puede encontrar en la respuesta de una API?

- Código de estado: indica si la operación fue exitosa o falló.
- Encabezados: metadatos como Content-Type, fecha, tamaño o información de caché.
- Cuerpo: los datos solicitados, normalmente en JSON, o un mensaje de error con su descripción.
- Tiempo de respuesta y tamaño: información útil para evaluar el rendimiento (la muestran herramientas como Postman).

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

- Permite probar los endpoints sin escribir código cliente.
- Ayuda a entender la estructura de las respuestas antes de integrarlas.
- Facilita detectar errores de la API de forma temprana.
- Permite guardar peticiones en colecciones y reutilizarlas.

9. ¿Qué es y para qué sirve Swagger UI?

Swagger UI es una herramienta que genera, a partir de una especificación OpenAPI, una documentación web interactiva de la API. Sirve para ver todos los endpoints disponibles, sus parámetros, cuerpos esperados y posibles respuestas, y además permite probar los endpoints directamente desde el navegador con el botón "Try it out".

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Postman es una plataforma para diseñar, probar, documentar y automatizar APIs. Su función principal es enviar peticiones HTTP a un servicio y mostrar la respuesta, lo que permite validar que la API funcione como se espera.



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?



- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si



Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.

The screenshot shows the Swagger Petstore API interface in a web browser. The URL bar displays `petstore3.swagger.io`. The main content area shows the Swagger Petstore - OpenAPI 3.0 specification. Below the specification, there are links for "Terms of service", "Contact the developer", "Apache 2.0", and "Find out more about Swagger".

On the right side, the Chrome DevTools network tab is open, showing a list of requests. The selected request is `openapijson`, which is a GET request to `https://petstore3.swagger.io/api/v3/openapijson`. The response status is `200 OK`. The response headers are visible, including `Access-Control-Allow-Headers`, `Access-Control-Allow-Methods`, `Access-Control-Allow-Origin`, and `Access-Control-Expose-Headers`.



Swagger UI

Pregunta a Gemini

petstore3.swagger.io/#/pet/getPetById

GET /pet/{petId} Find pet by ID.

Returns a single pet.

Parameters

Try it out

Name	Description
petId * required integer(\$int64) (path)	ID of pet to return petId

Responses

Code	Description	Links
200	successful operation Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 10, "name": "doggie", "category": { "id": 1, "name": "Dogs" } }</pre>	No links